

Plataforma SaaS de Rastreo y Seguimiento de Encomiendas — Ruteo

Contexto

El proyecto ruteo es un desarrollo desde cero (greenfield). El objetivo es construir una plataforma **SaaS multi-tenant** que se pueda ofrecer a distintos negocios de mensajería/paquetería. Cada negocio (tenant) gestiona sus envíos, rutas, repartidores y clientes de forma aislada.

Estrategia acordada:

- **Primero el backend** con NestJS + PostgreSQL (API + lógica de negocio).
- **Luego los clientes:** panel web (admin/operadores/comercios), app móvil de clientes finales y app móvil de repartidores.

Este documento define **qué funcionalidades** debe tener la plataforma completa y **en qué orden** construirlas, empezando por el backend.

Enfoque prioritario — envío internacional USA -> Honduras: varios clientes son empresas que envían desde USA hacia Honduras. La funcionalidad estrella es que el cliente entre a la app y vea **por dónde viene su paquete** a lo largo de todo el trayecto internacional (bodega en USA -> consolidación -> tránsito aéreo/marítimo -> aduana Honduras -> bodega HN -> reparto local -> entregado), con mapa, hitos y ETA en tiempo real. Esto exige modelar el envío como un viaje de ****múltiples tramos (legs)**** internacionales, no solo un reparto local de última milla.

1. Catálogo de funcionalidades (por rol)

A. Multi-tenancy y plataforma SaaS (transversal)

- Registro y aislamiento de tenants (cada negocio con sus datos separados).
- Planes de suscripción y límites por plan (nº de envíos, usuarios, API calls).
- Facturación/billing del SaaS (Stripe u otro) y control de estado de cuenta.
- Marca personalizable por tenant (logo, colores, dominio, página pública de rastreo).
- Roles y permisos (RBAC): owner, admin, operador, repartidor, comercio, soporte.
- Auditoría de acciones (quién cambió qué y cuándo).

B. Operadores / Admin (panel web)

- Dashboard con métricas: envíos activos, entregados, en ruta, incidencias, SLA.
- Gestión de envíos: crear, editar, cancelar, buscar/filtrar, importar por lote (CSV).
- Gestión de rutas: crear rutas, asignar repartidores, optimización de paradas.
- Asignación automática/manual de repartidores por zona o carga.
- Gestión de zonas de cobertura y tarifas (por peso, distancia, zona).
- Gestión de flota y repartidores (altas, disponibilidad, horarios).
- Gestión de incidencias/devoluciones y reprogramaciones.
- Soporte: ver historial completo de un envío, timeline de eventos.
- Reportes exportables (CSV/PDF) y analítica operativa.

C. Comercios / Remitentes (panel web + API)

- Crear envíos individuales y por lote.
- Generar e imprimir etiquetas con código de barras / QR.
- Integración por API pública (webhooks de estados, creación de envíos).
- Cotizador de envíos (precio estimado por destino/peso).
- Historial y estado de sus envíos; conciliación de cobros (COD).

D. Repartidores (app móvil)

- Login y lista de envíos/paradas asignadas del día.
- Ruta optimizada con navegación GPS (mapa + orden de paradas).
- Escaneo de código de barras/QR para recoger y entregar.
- Actualización de estados: recogido, en ruta, entregado, fallido.
- Prueba de entrega (POD): firma digital, foto, geolocalización, motivo de fallo.
- Cobro contra entrega (COD) y registro de pago.
- Modo offline con sincronización posterior.
- Tracking de ubicación en tiempo real hacia el backend.

E. Clientes finales (app móvil + página web pública)

- Rastreo por número de guía/tracking (sin login o con login).
- **"¿Por dónde viene mi paquete?":** mapa con la ubicación/tramo actual del envío y línea de tiempo de hitos del trayecto internacional (USA -> Honduras).
- Timeline de estados en tiempo real + mapa de ubicación del paquete.
- Notificaciones push/SMS/email en cada cambio de estado o cruce de hito.
- Ventana estimada de entrega (ETA) por tramo y ETA final a domicilio.
- Instrucciones de entrega y contacto con el repartidor.
- Historial de envíos recibidos y calificación del servicio.

F. Envío internacional USA -> Honduras (funcionalidad estrella)

- **Casillero virtual:** cada cliente recibe una dirección en USA para recibir compras; al llegar a bodega USA se registra y se notifica ("pre-alerta").
- **Consolidación:** agrupar varios paquetes de un cliente en un solo envío.
- **Viaje por tramos (legs):** el envío avanza por etapas geográficas con hito, ubicación, timestamp y ETA por cada tramo:
`recibido en bodega USA -> consolidado -> en tránsito (aéreo/marítimo) -> en aduana Honduras -> liberado de aduana -> en bodega HN -> en ruta (última milla) -> entregado`.
- **Seguimiento del transportista internacional:** guardar el tracking number del carrier (courier/aerolínea/naviera) e integrar/consultar su estado cuando sea posible.
- **Mapa del trayecto:** mostrar origen, destino y ubicación/tramo actual sobre mapa.
- **Aduana e impuestos:** registro de estado aduanal, declaración de valor, cálculo/cobro de impuestos y aranceles antes de liberar.
- **Notificaciones por hito clave:** llegó a USA, salió de USA, en aduana, liberado, en Honduras, en reparto, entregado.
- **Facturación de flete internacional** por peso/volumen + tarifas por libra.

2. Dominio y modelo de datos (núcleo del backend)

Entidades principales (PostgreSQL, con tenant_id en todas para aislamiento):

- **Tenant** — negocio suscrito (plan, branding, estado).
- **User** — usuario con rol dentro de un tenant.
- **Merchant / Sender** — comercio remitente.
- **Customer** — destinatario/cliente final.
- **Shipment** — envío (origen, destino, peso, dimensiones, estado, tracking_number, COD, valor declarado, tipo: internacional/local).
- **ShipmentLeg** — tramo del viaje internacional (secuencia, origen, destino, medio: aéreo/marítimo/terrestre, estado del tramo, carrier, tracking externo, ETA).
- **ShipmentEvent** — cada cambio de estado/hito (timeline inmutable, con geo + timestamp + actor + tramo asociado).
- **VirtualLocker** — casillero: dirección USA asignada al cliente para recibir compras.
- **Consolidation** — agrupación de varios paquetes en un envío.
- **CustomsRecord** — estado aduanal, declaración de valor, impuestos/aranceles.
- **Carrier** — transportista internacional (aerolínea/naviera/courier) y su API/tracking.
- **Route** — ruta de un repartidor (fecha, lista ordenada de paradas).
- **RouteStop** — parada de una ruta ligada a un shipment.
- **Driver / Courier** — repartidor (disponibilidad, vehículo, zona).
- **Zone** — zona de cobertura.
- **Rate / Tariff** — tarifa por zona/peso/distancia.
- **ProofOfDelivery** — firma, fotos, motivo, geo.
- **Payment** — cobros (COD y suscripción SaaS).
- **Notification** — registro de notificaciones enviadas.
- **Webhook / ApiKey** — integraciones por tenant.
- **AuditLog** — auditoría.

Estados de un Shipment **internacional** (máquina de estados por hitos):

`created -> received_usa -> consolidated -> in_transit_intl ->
in_customs_hn -> customs_cleared -> in_warehouse_hn -> out_for_delivery ->
delivered con ramas failed_attempt, returned, cancelled, on_hold_customs`.

Estados de un Shipment **local** (última milla):

`created -> label_generated -> picked_up -> in_transit -> out_for_delivery ->
delivered con ramas failed_attempt, returned, cancelled`.

3. Arquitectura del backend (NestJS + PostgreSQL)

- **Stack:** NestJS (TypeScript), PostgreSQL, TypeORM o Prisma como ORM, Redis (cache + colas), BullMQ para trabajos asíncronos.
- **Estructura modular** (un módulo Nest por dominio):
auth, tenants, users, shipments, tracking, routes, drivers,
zones, rates, notifications, payments, webhooks, billing, reports.
- **Multi-tenancy:** estrategia **shared schema** con tenant_id + guard/interceptor que inyecta y filtra por tenant en cada request (o Row-Level Security en Postgres).
- **Autenticación:** JWT (access + refresh), guards por rol (RBAC), API keys para integraciones de comercios.
- **Tiempo real:** WebSocket gateway (Socket.IO) para tracking en vivo del repartidor y actualizaciones de estado a clientes.
- **Colas/eventos:** BullMQ para notificaciones, webhooks, optimización de rutas, importación por lote.
- **Geo:** extensión PostGIS para zonas/distancias; integración con proveedor de

mapas (Google Maps / Mapbox) para geocoding, rutas y ETA.

- **Documentación API:** OpenAPI/Swagger autogenerada.
- **Observabilidad:** logs estructurados, health checks, métricas.

Estructura de carpetas propuesta

```
ruteo/  
  backend/                # NestJS  
    src/  
      common/             # guards, interceptors, decorators (tenant, roles)  
      config/  
      modules/  
        auth/ tenants/ users/ shipments/ tracking/ routes/  
        drivers/ zones/ rates/ notifications/ payments/  
        webhooks/ billing/ reports/  
        lockers/ consolidation/ customs/ carriers/    # internacional USA->HN  
      main.ts  
    test/  
  web/                    # panel admin/operadores/comercios (fase 2)  
  mobile-customer/        # app clientes (fase 2)  
  mobile-courier/         # app repartidores (fase 2)
```

4. Fases de implementación (backend primero)

Fase 0 — Cimientos del backend

- Inicializar proyecto NestJS, configuración, conexión PostgreSQL, ORM, migraciones.
- Docker Compose (Postgres + Redis) para entorno local.
- Módulo auth (JWT, refresh, RBAC) + módulo tenants (multi-tenancy y guard).

Fase 1 — Núcleo de envíos y rastreo (incluye tramos internacionales)

- Módulo shipments (CRUD, tracking_number, máquina de estados internacional/local).
- Módulo tracking con **ShipmentLeg** y **ShipmentEvent** (timeline de hitos + endpoint público de rastreo "¿por dónde viene mi paquete?").
- WebSocket gateway para estados/hitos en tiempo real.
- Swagger de la API.

Fase 1.5 — Internacional USA -> Honduras (funcionalidad estrella)

- Módulo lockers (casillero virtual + pre-alerta al recibir en bodega USA).
- Módulo consolidation (agrupar paquetes).
- Módulo customs (estado aduanal, valor declarado, impuestos/aranceles).
- Módulo carriers (transportistas internacionales + tracking externo).
- Mapa del trayecto y ETA por tramo en el endpoint de rastreo.

Fase 2 — Operación logística

- Módulos drivers, zones, rates, routes (asignación + optimización básica).
- POD (firma/foto), estados de recogida y entrega.
- Importación por lote y generación de etiquetas (código de barras/QR).

Fase 3 — Notificaciones, pagos e integraciones

- Módulo notifications (push/SMS/email vía colas).
- Módulo payments (COD) + billing (suscripción SaaS con Stripe).
- Módulo webhooks + API keys para comercios.

Fase 4 — Analítica y reportes

- Dashboards de métricas, SLA, reportes exportables.

Fase 5 — Clientes (frontend)

- Panel web (React/Next.js) para admin/operadores/comercios.
- App móvil de clientes (rastreo, notificaciones) — React Native/Flutter.
- App móvil de repartidores (GPS, escaneo, POD, offline) — React Native/Flutter.

5. Verificación

- **Backend:** pruebas unitarias (Jest) por módulo + pruebas e2e de NestJS.
- Levantar docker compose up y verificar migraciones aplicadas.
- Probar flujo end-to-end por Swagger/Postman:

crear tenant -> login -> crear envío -> cambiar estados -> rastreo público refleja timeline.

- **Flujo internacional USA -> Honduras:** recibir en bodega USA (pre-alerta) -> consolidar -> tránsito -> aduana -> liberado -> bodega HN -> reparto -> entregado, y verificar que el rastreo público muestra el tramo actual y el mapa/ETA.

- Validar aislamiento multi-tenant: un tenant no puede leer datos de otro.
- WebSocket: verificar que un cambio de estado emite evento en tiempo real.

6. Decisiones abiertas para confirmar antes de codificar

- ORM: **Prisma** vs **TypeORM**.
- Multi-tenancy: **shared schema + tenant_id** vs **schema-per-tenant** vs **RLS**.
- Proveedor de mapas/rutas: Google Maps vs Mapbox vs OSRM (open source).
- Stack de las apps móviles: React Native vs Flutter.
- Proveedor de notificaciones: Firebase (push), Twilio (SMS), correo (SES/SendGrid).
- Transporte internacional: ¿flete aéreo, marítimo o ambos? ¿carriers específicos con API de tracking, o entrada manual de hitos por el operador?
- Casillero: ¿integración con bodega física en USA (escaneo al recibir) o registro manual?
- Aduana: reglas de cálculo de impuestos/aranceles de Honduras a aplicar.